



# Concurrency in Swift

Mohammed Mubashir Meddekar  
iOS Developer

Evolution of Swift Concurrency

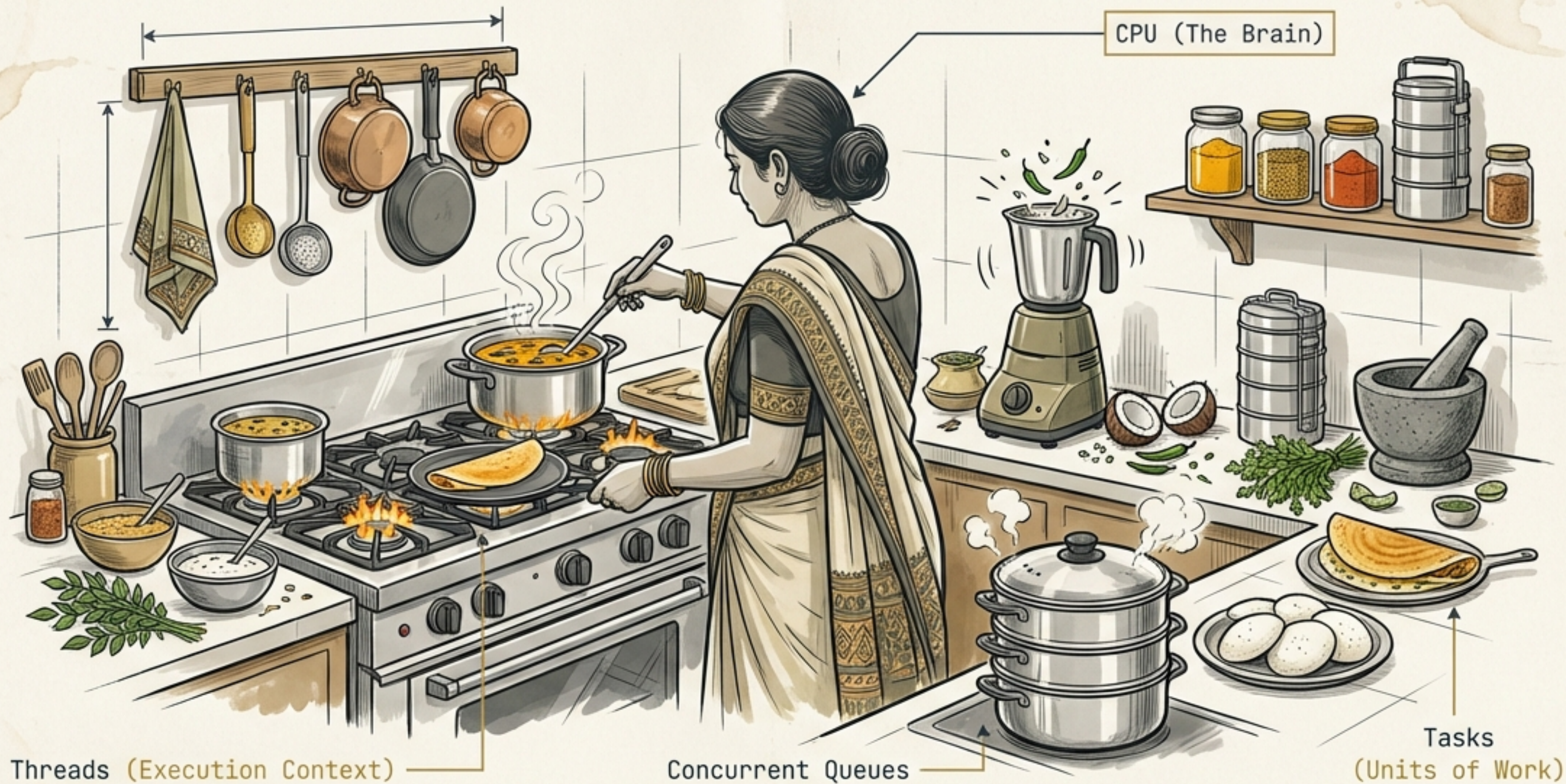
Concurrency in Swift 6

Common Concurrency mistakes

Quick Comparison

Swift Concurrency Best Practices

# The Grand Indian Kitchen



She manages steaming idlis, grinding chutney, and flipping dosas simultaneously without crashing. That is Concurrency.

# Swift Concurrency



Make concurrent tasks easier to write correctly

Make concurrent tasks easier to write correctly

Identify where concurrency is introduced and what data is shared

Make concurrent tasks easier to write correctly

Identify where concurrency is introduced and what data is shared

Protect against data races

# Evolution of Swift Concurrency

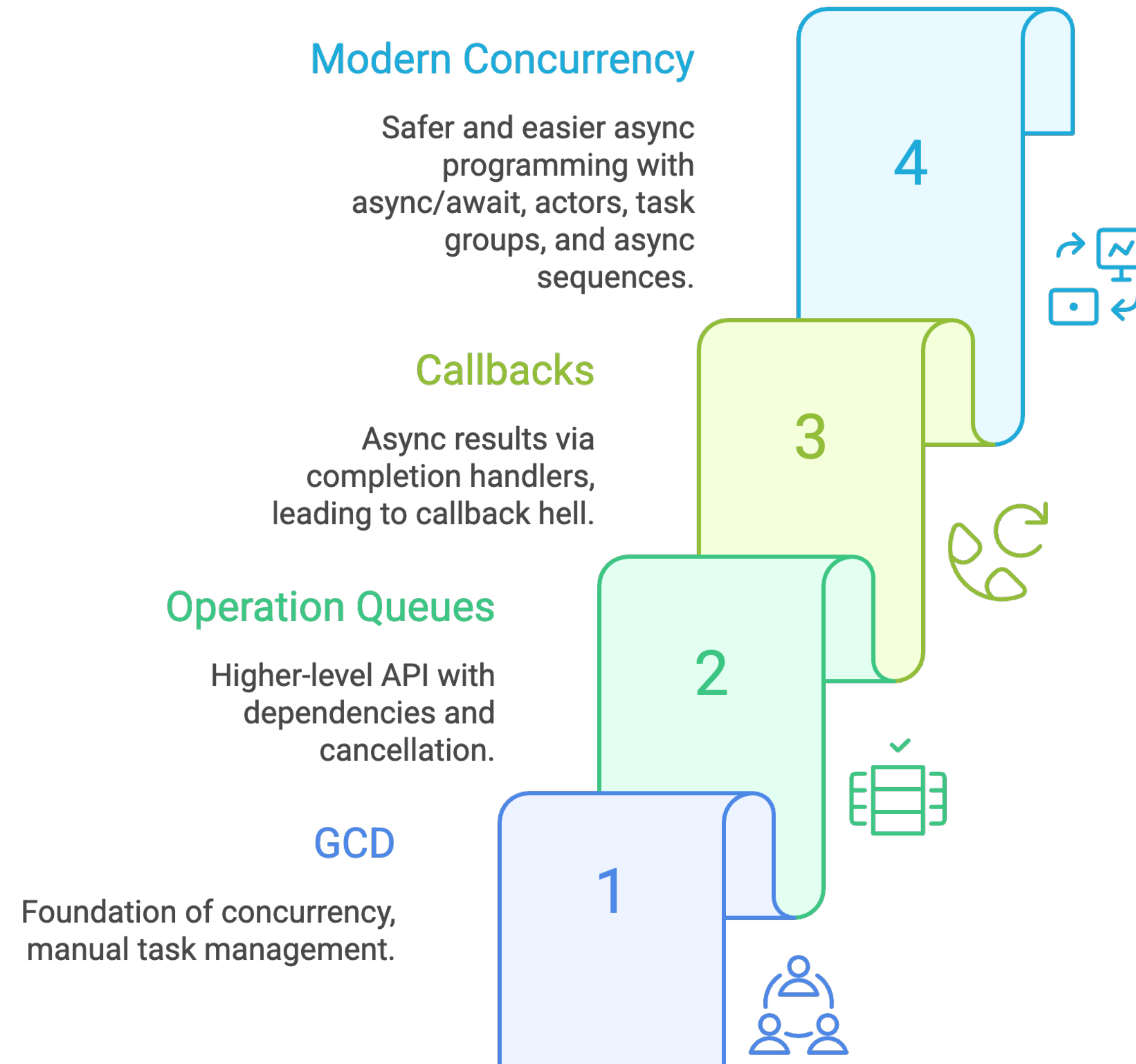
Concurrency in Swift 6

Common Concurrency mistakes

Quick Comparison

Swift Concurrency Best Practices

# Evolution of Swift Concurrency



Evolution of Swift Concurrency

**Concurrency in Swift 6**

Common Concurrency mistakes

Quick Comparison

Swift Concurrency Best Practices

Single-threaded code

Single-threaded code

**Asynchronous tasks**

Single-threaded code

Asynchronous tasks

**Introducing Concurrency**

Single-threaded code

Asynchronous tasks

Introducing Concurrency

**Sharing data**

Single-threaded code

Asynchronous tasks

Introducing Concurrency

Sharing data

**Actors**

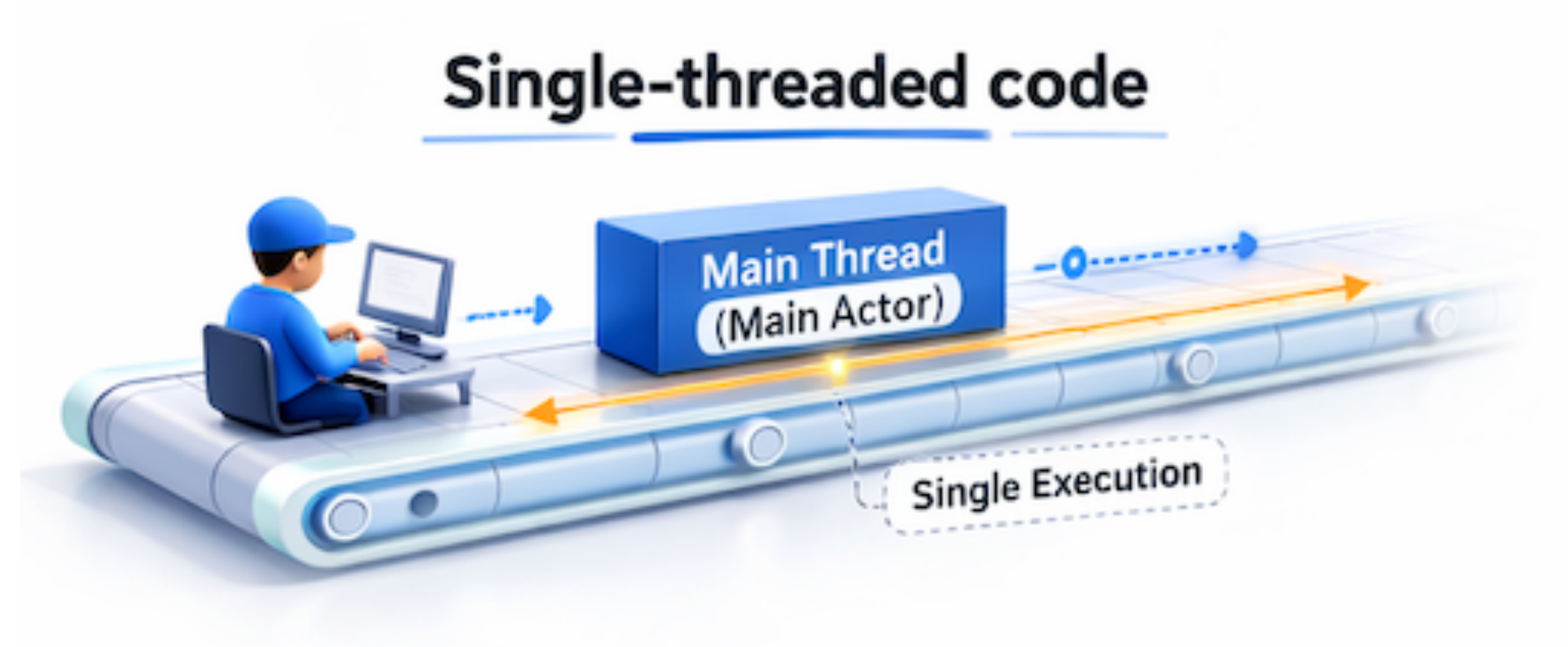
Single-threaded code

Asynchronous tasks

Introducing Concurrency

Sharing data

Actors



Single-threaded code runs on the main thread

Single-threaded code runs on the main thread

There is no concurrency on the main actor

Single-threaded code runs on the main thread

There is no concurrency on the main actor

Data or code can be specified on the main actor using the  
`@MainActor`

# Single-threaded synchronous UI code



```
func fetchAndDisplayImage(url: URL) throws {  
    let data = try Data(contentsOf: url)  
    let image = decodeImage(data)  
    view.displayImage(image)  
}
```

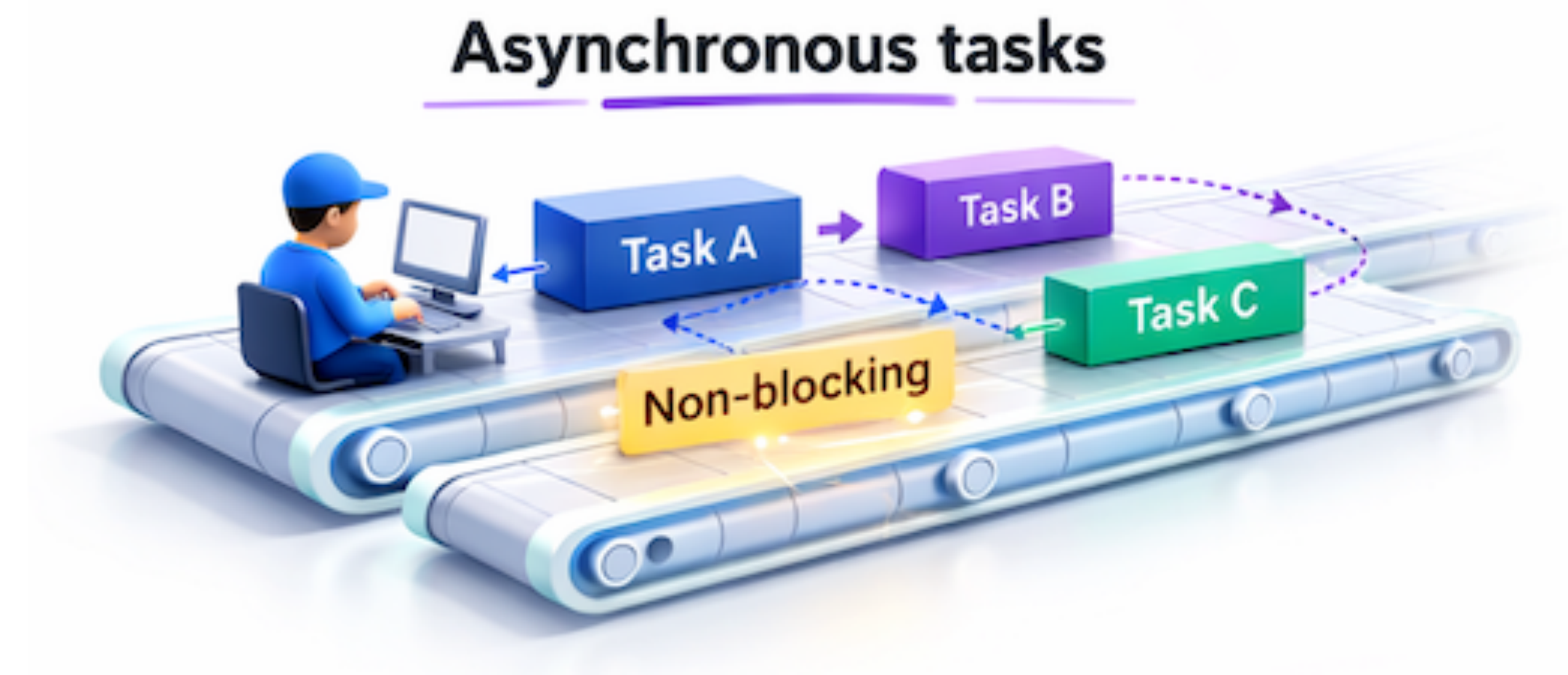
Single-threaded code

**Asynchronous tasks**

Introducing Concurrency

Sharing data

Actors



Swift concurrency uses `async` and `await` to make functions non-blocking

Swift concurrency uses `async` and `await` to make functions non-blocking

Prevents hangs and improves UI responsiveness

Swift concurrency uses `async` and `await` to make functions non-blocking

Prevents hangs and improves UI responsiveness

Runs before and after the awaited event

# Migration - GCD Closures to async/await

## ✗ Before — Nested closures + DispatchQueue

```
func showVideoMaterial(trvm: TalkResourceVM) {
    guard Thread.isMainThread else {
        DispatchQueue.main.async {
            self.showVideoMaterial(trvm: trvm)
        }
    }
    source.present(vc, animated: true) {
        guard let strongSelf = self else { return }
        let stream = vmVC.viewModel.videoStreamObj(...)
        vc.setupPlayer(...) // callback pyramid
    }
}
```

### Problems with GCD

- Thread.isMainThread guard + re-dispatch
- Nested completion handlers
- Manual self capture management

## ✓ After — Modern async/await

```
@MainActor
func showVideoMaterial(trvm: TalkResourceVM) async {
    addLoadingView?()

    let stream = vmVC.viewModel.videoStreamObj(...)
    let playerVC = ZSTVRSwiftUIVideoViewerKit(
        stream: stream, chapters: chapters, subtitles: subtitles
    )
    await source.present(playerVC) // linear, readable
}
```

### Why async/await wins

- @MainActor replaces DispatchQueue check
- Flat, linear code flow
- Compiler-enforced thread safety

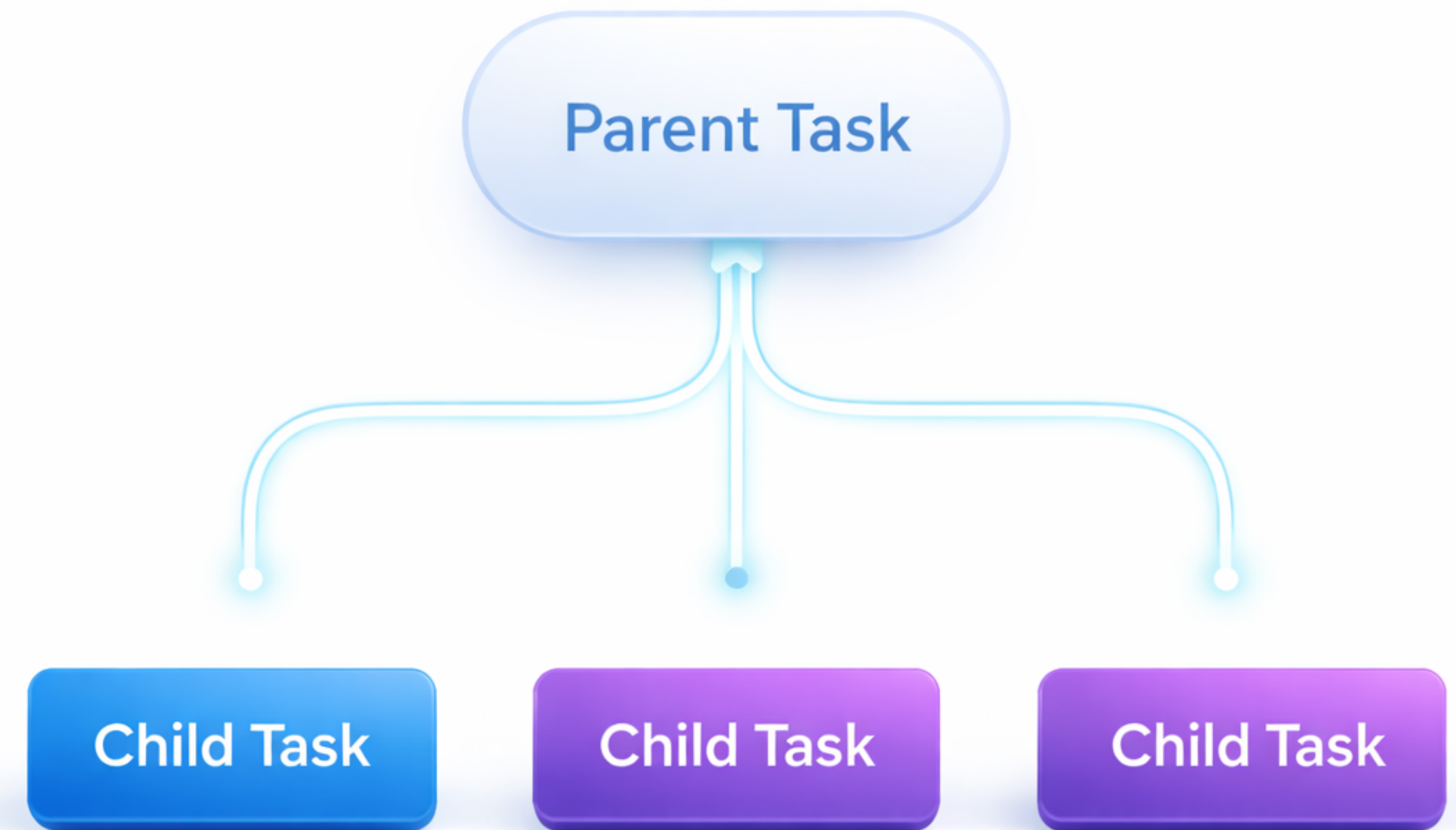
Single-threaded code

Asynchronous tasks

**Introducing Concurrency**

Sharing data

Actors



Concurrency allows parts of code to run on a background thread

Concurrency allows parts of code to run on a background thread

Run code on background threads

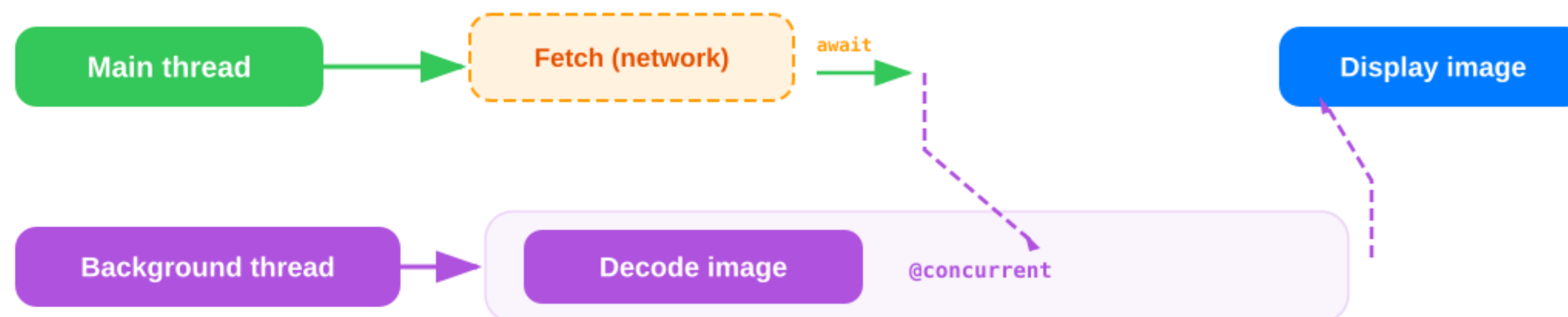
Concurrency allows parts of code to run on a background thread

Run code on background threads

Freeing up main thread

# Introducing concurrency

Decode image runs on a background thread, freeing the main thread



```
func fetchAndDisplayImage(url: URL) async throws {  
    let (data, _) = try await URLSession.shared.data(from: url)  
    let image = await decodeImage(data) // @concurrent – runs in background  
    view.displayImage(image)  
}
```



**Swift Compiler - Concurrency**

Setting

**Default Actor Isolation**

**nonisolated**



@concurrent attribute, Swift is instructed to run a function in the background

@concurrent attribute, Swift is instructed to run a function in the background

'nonisolated' keyword allows clients to choose where to run code

@concurrent attribute, Swift is instructed to run a function in the background

'nonisolated' keyword allows clients to choose where to run code

Concurrent thread pool manages scheduling the work on threads

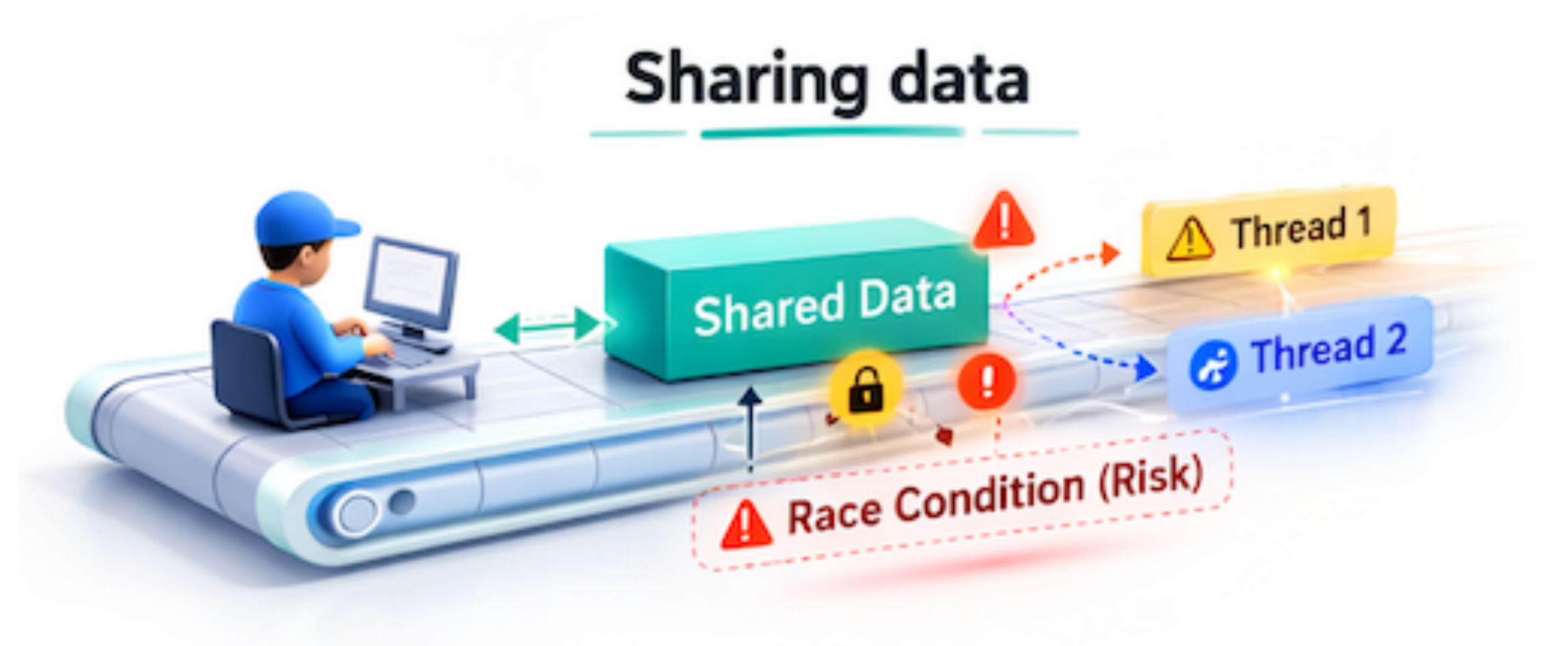
Single-threaded code

Asynchronous tasks

Introducing Concurrency

Sharing data

Actors



Swift's design helps mitigate this issue by providing compile-time checks

Swift's design helps mitigate this issue by providing compile-time checks

Value types that conform to the 'Sendable' are always safe to share concurrently

Swift's design helps mitigate this issue by providing compile-time checks

Value types that conform to the 'Sendable' are always safe to share concurrently

Only Sendable types are shared between actors and tasks

✓ Value types — automatically Sendable

```
// Structs with value-type properties are implicitly Sendable
struct UserProfile: Sendable {
    let name: String
    let email: String
    let joinDate: Date
}

// Safe to send across actors — each task gets its own copy
let profile = UserProfile(name: "Mubashir", ...)
await backgroundActor.save(profile) // ✓ copy sent
await analyticsActor.track(profile) // ✓ another copy

// String, Int, Double, Date, Array<Sendable>, structs → all safe
// No shared mutable state — no data race possible
```

**Value types = safe by default**

Copied on assignment — each task has independent data. No sharing.

▲ Reference types — require explicit conformance

```
// ✗ Classes are NOT Sendable by default
class ImageCache {
    var cache: [String: UIImage] = [:] // mutable state!
}

// ✔ Fix 1: Use actor instead
actor SafeImageCache {
    private var cache: [String: UIImage] = [:]
    func store(_ image: UIImage, forKey key: String) {
        cache[key] = image
    }
}

// ✔ Fix 2: @Sendable closure — captures only Sendable values
Task { @Sendable in // compiler checks captures
    await cache.store(image, forKey: url.absoluteString)
```

Reference types = shared state

Classes share the same memory.  
Use actor to serialize access, or  
make immutable (let only) + Sendable.

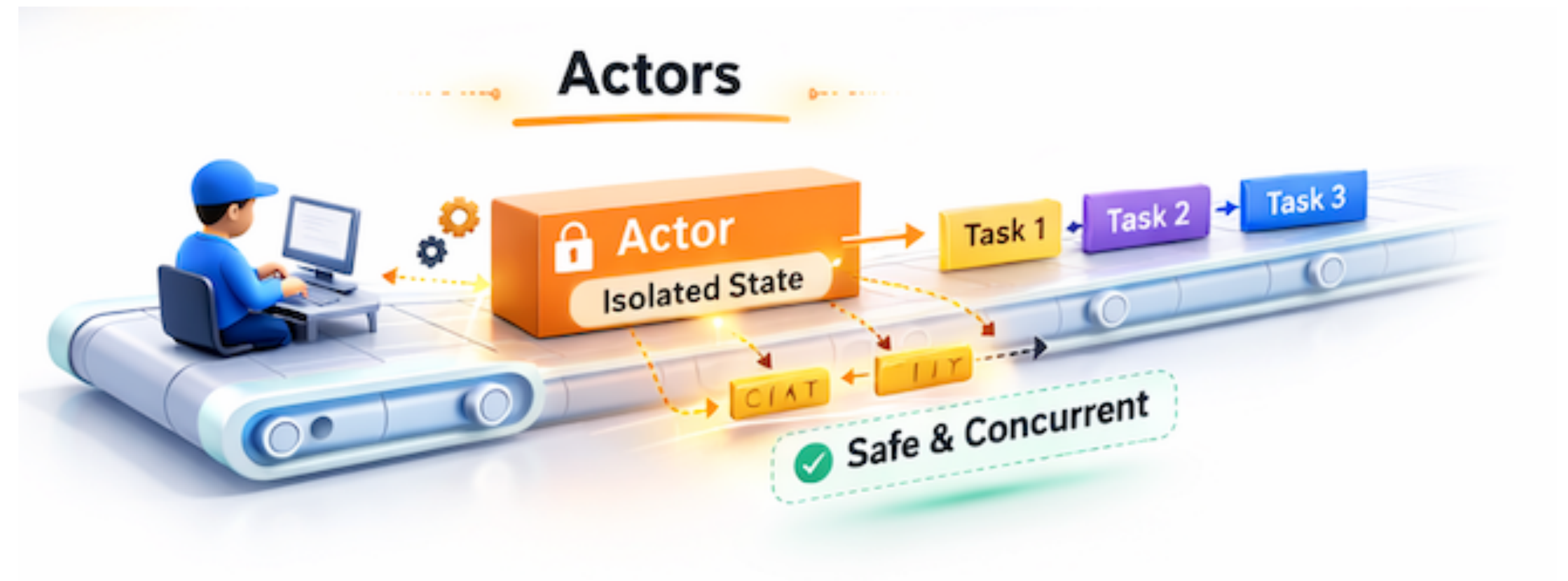
Single-threaded code

Asynchronous tasks

Introducing Concurrency

Sharing data

Actors



Actors isolate their data, allowing only one thread to access it at a time

Actors isolate their data, allowing only one thread to access it at a time

Moves state from the main actor to dedicated actors

Actors isolate their data, allowing only one thread to access it at a time

Moves state from the main actor to dedicated actors

Frees up the main thread to maintain responsiveness

```
class Counter {
    var value = 0

    func increment() {
        value += 1
    }
}

func runCounter() async {
    let counter = Counter()
    await withTaskGroup(of: Void.self) { group in
        for _ in 0..<5 {
            group.addTask {
                counter.increment() // ⚠️ Not concurrency safe
            }
        }
    }
    print(counter.value)
}
```

```
actor SafeCounter {
    private(set) var value = 0

    func increment() {
        value += 1
    }
}

func runCounter() async {
    let counter = SafeCounter()
    await withTaskGroup(of: Void.self) { group in
        for _ in 0..<5 {
            group.addTask {
                await counter.increment() // ✅ Safe with actor
            }
        }
    }
    print(await counter.value)
}
```

Evolution of Swift Concurrency

Concurrency in Swift 6

**Common Concurrency mistakes**

Quick Comparison

Swift Concurrency Best Practices

Forgetting `[weak self]` inside `DispatchQueue.async`

Forgetting [weak self] inside DispatchQueue.async

**Blocking the Main Thread**

Forgetting [weak self] inside DispatchQueue.async

Blocking the Main Thread

**Data Races**

Forgetting [weak self] inside DispatchQueue.async

Blocking the Main Thread

Data Races

**Not Cancelling Tasks Properly**

Forgetting [weak self] inside DispatchQueue.async

Blocking the Main Thread

Data Races

Not Cancelling Tasks Properly

**Ignoring Sendable Warnings**

Forgetting [weak self] inside DispatchQueue.async

Blocking the Main Thread

Data Races

Not Cancelling Tasks Properly

Ignoring Sendable Warnings

# Real-world: Capture [weak self] in async closures

From a production video player — AudienceEngagement tracking

## ✗ Before — Strong self captured

```
vc.onStateChange = { // ! strong capture of self
    (status, stateStartIndex, stateEndIndex) in
    let model = AudienceEngagementRequestModal()
    model.talkId = self.sessionData.getUpcomingTalk()?.talkId
    model.status = status.rawValue
    model.stateStartIndex = stateStartIndex
    self.lessonViewModel?.sendAudEngagement(model: model)
}
```

## ▲ Memory leak risk

ViewController never deallocates if  
closure retains strong reference to self

## ✓ After — [weak self] + guard

```
vc.onStateChange = { [weak self] (status, start, end) in
    guard let strongSelf = self,
        let sessionData = strongSelf.lessonViewModel?.sessionData
    else { return }

    let model = AudienceEngagementRequestModal()
    model.talkId = sessionData.getUpcomingTalk()?.talkId
    model.talkResourceId = trvm.talkResourceId
    model.status = status.rawValue
    strongSelf.lessonViewModel?.sendAudEngagement(model: model)
}
```

## ✓ Safe pattern

guard let self = self ensures early exit  
if VC is deallocated during async work

Forgetting [weak self] inside DispatchQueue.async

**Blocking the Main Thread**

// ❌ Common culprits

let data = try! Data(contentsOf: remoteURL) // Synchronous network call!

let image = UIImage(data: hugeImageData) // Heavy decoding on main

let results = coreDataContext.fetch(fetchRequest) // Unoptimised Core Data

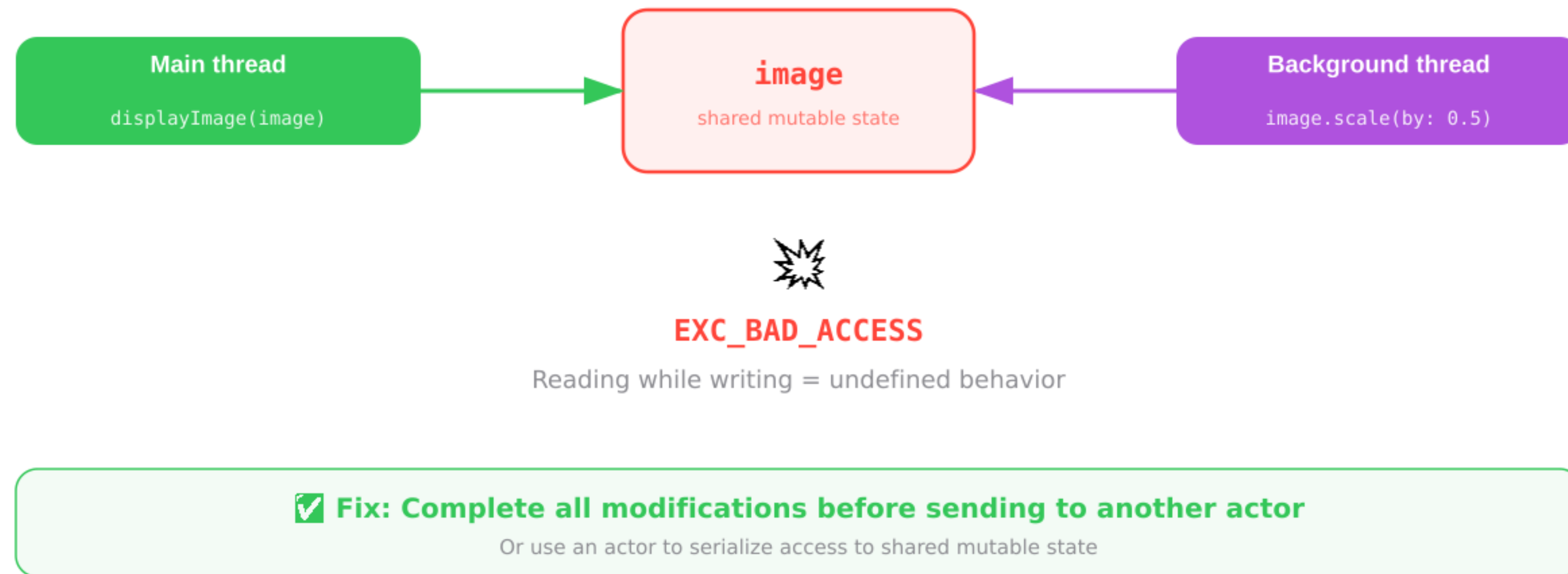
DispatchQueue.main.sync { ... } // Deadlock from main thread

Forgetting [weak self] inside DispatchQueue.async

Blocking the Main Thread

**Data Races**

## Data race — concurrent access to shared state



```
// ❌ Classic data race
var count = 0
DispatchQueue.concurrentPerform(iterations: 1000) { _
    in
        count += 1 // Multiple threads reading and
writing simultaneously
}
print(count) // Could be anything from 900 to 1000
```

```
// ✅ Fix with Actor (modern)
actor Counter {
    private var count = 0
    func increment() { count += 1 }
    func value() -> Int { count }
}

// ✅ Fix with Dispatch Barrier (GCD)
let queue = DispatchQueue(label: "counter",
attributes: .concurrent)
queue.async(flags: .barrier) { count += 1 }
```

Forgetting [weak self] inside DispatchQueue.async

Blocking the Main Thread

Data Races

**Not Cancelling Tasks Properly**

# Task cancellation — stop tracking on dismiss

From SessionDetailViewController — engagement timer cleanup

## ✗ Fire and forget — timer leaks

```
// Timer keeps firing even after VC is dismissed
func startEngagementTracking() {
    Timer.scheduledTimer(withTimeInterval: 3.0, repeats: true) {
        self.sendEngagement() // self captured strongly!
    }
}
```

## ▲ Timer retains self

VC can't dealloc → memory leak + API spam

## ✓ Track and cancel — clean lifecycle

```
private var engagementTimer: Timer?

func startEngagementTracking() {
    engagementTimer = Timer.scheduledTimer(
        withTimeInterval: 3.0, repeats: true
    ) { [weak self] _ in
        guard let self else { return }
        self.engagementDelegate?.videoPlayer(self, ...)
    }
}

func stopEngagementTracking() { // Called in viewWillDisappear
    engagementTimer?.invalidate()
    engagementTimer = nil
}
```

## ✓ Lifecycle-aware tracking

- [weak self] prevents retain cycle
- invalidate() in viewWillDisappear
- Clean deallocation guaranteed

Forgetting [weak self] inside DispatchQueue.async

Blocking the Main Thread

Data Races

Not Cancelling Tasks Properly

**Ignoring Sendable Warnings**

```
import Foundation

// Value types are Sendable

extension URL: Sendable {}

// Collections of Sendable elements

extension Array: Sendable where Element: Sendable {}

// Structs and enums with Sendable storage

struct ImageRequest: Sendable {

    var url: URL

}

// Main-actor types are implicitly Sendable

@MainActor class ImageModel {}
```

```
// ❌ The lazy escape

class MyModel: @unchecked Sendable { // Silences the
    compiler, not the bug

    var data: [String] = [] // Still a data race
    waiting to happen

}

// ✅ Make it truly safe

actor MyModel { // or use struct/class with proper
    synchronization

    var data: [String] = []

}
```

Evolution of Swift Concurrency

Concurrency in Swift 6

Common Concurrency mistakes

**Quick Comparison**

Swift Concurrency Best Practices

| ❌ Novice Pattern                        | ✅ Expert Pattern                        |
|-----------------------------------------|-----------------------------------------|
| Strong self in async closures           | [weak self] + guard let self            |
| Sync network calls on main thread       | All I/O on background threads/async     |
| Shared mutable state without protection | Actors or dispatch barriers             |
| DispatchQueue.main.sync from main       | DispatchQueue.main.async or @MainActor  |
| Unlimited concurrent dispatches         | Controlled concurrency with TaskGroup   |
| Ignoring Task cancellation              | Store tasks, cancel on deinit/disappear |
| @unchecked Sendable everywhere          | Proper Sendable conformance or actors   |
| Nested callback pyramids                | async/await with structured concurrency |
| No error handling in async code         | try await with proper catch blocks      |
| Testing on simulator only               | Thread Sanitizer + real device testing  |

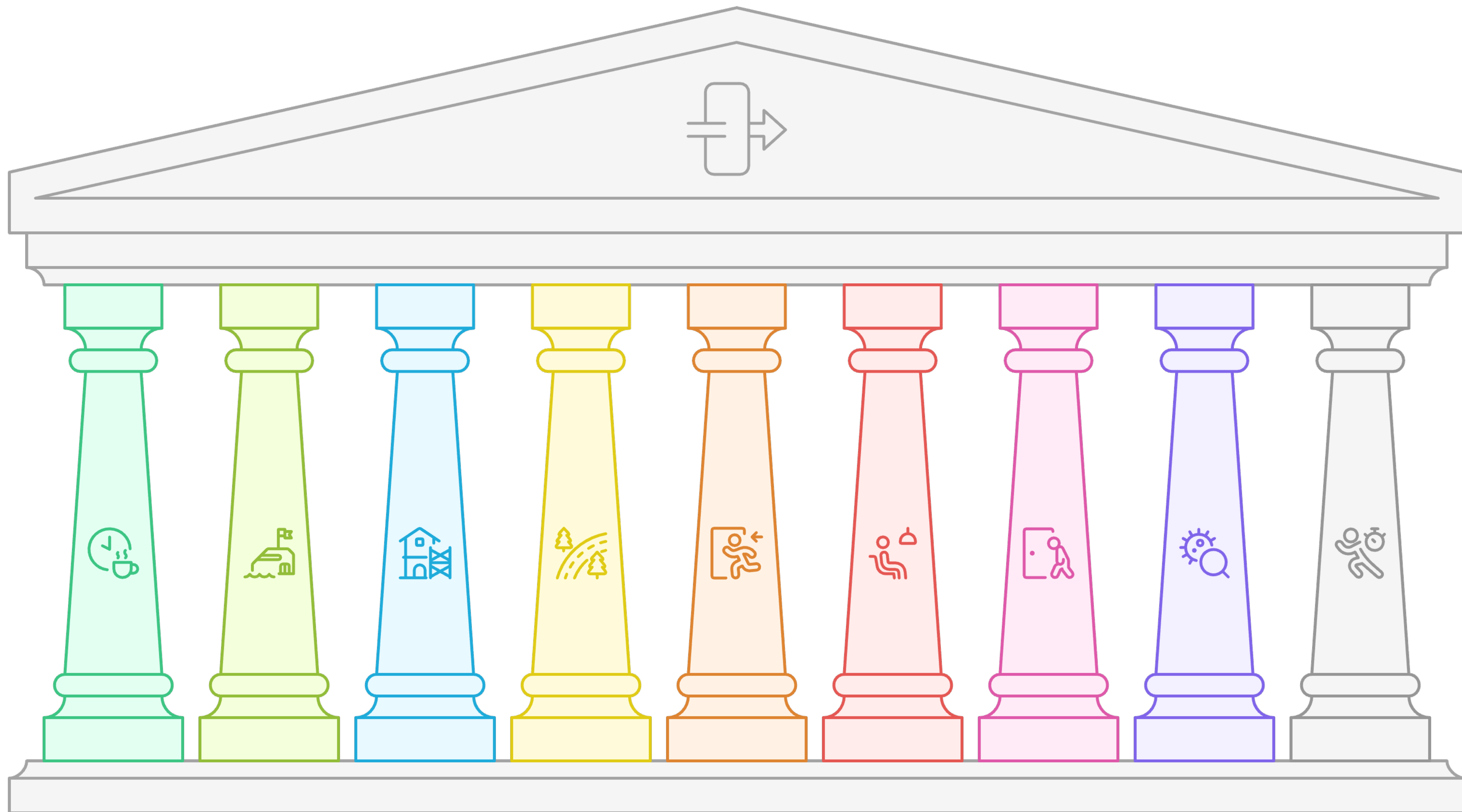
Evolution of Swift Concurrency

Concurrency in Swift 6

Common Concurrency mistakes

Quick Comparison

**Swift Concurrency Best Practices**



### async/await

Write asynchronous code that looks synchronous.

### Actors

Safely manage shared mutable state.

### Task Groups

Structure and manage parallel tasks effectively.

### Main Thread

Keep the main thread free for UI responsiveness.

### Error Handling

Handle errors gracefully using try/await.

### Concurrency Avoidance

Avoid overusing concurrency to maintain simplicity.

### Cancellation Support

Provide mechanisms to cancel ongoing tasks.

### Testing

Write comprehensive tests for concurrent code.

### Optimization

Profile and optimize concurrent code using Instruments.

*You don't have to do everything at once.  
You just have to do the right thing at the right time.*

**Thank you, Chennai. Thank you, Swift** 